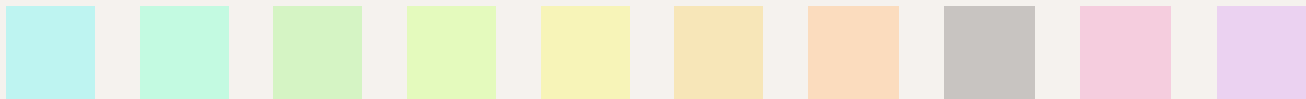


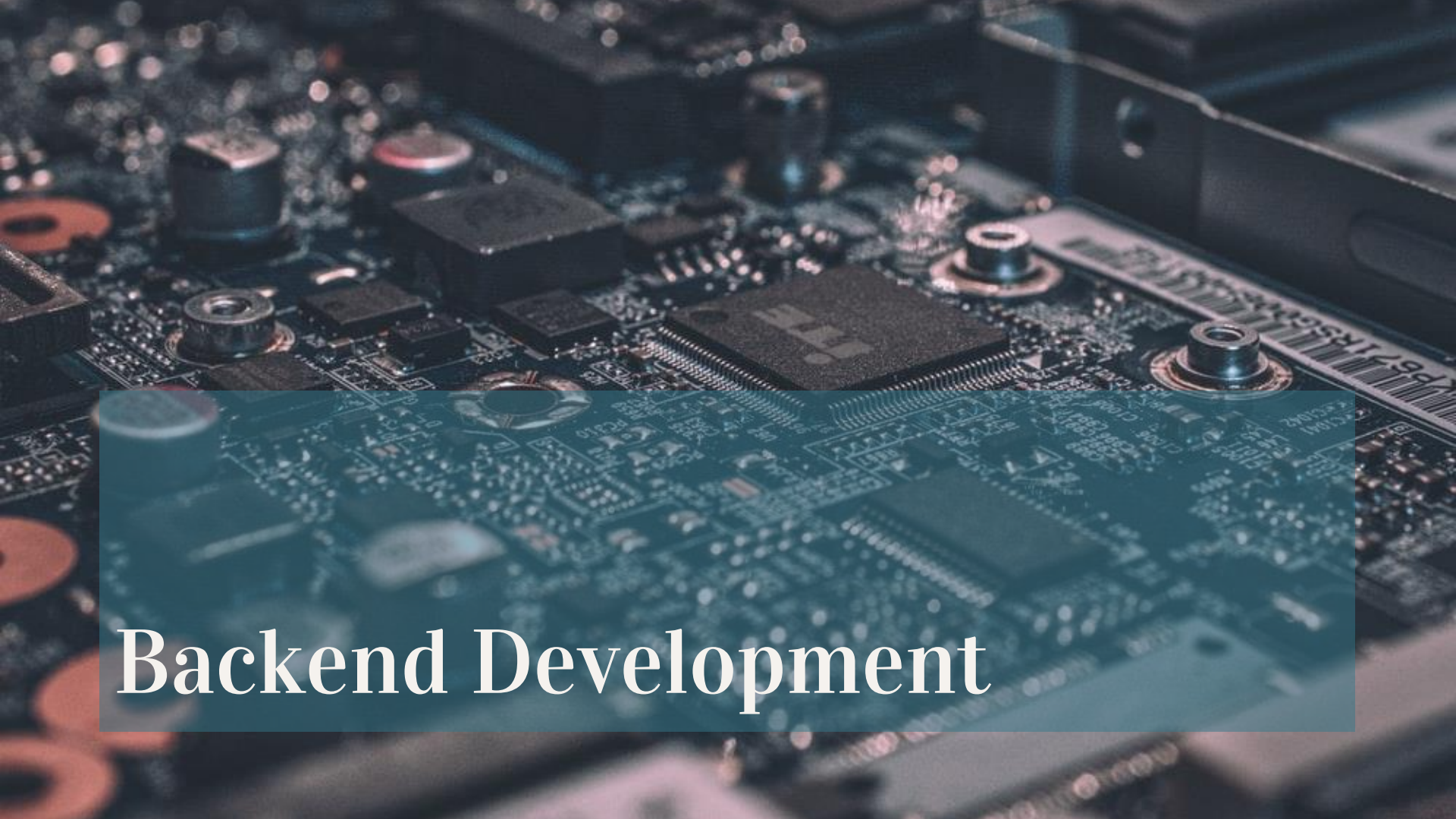
Server Side

מבוא לבניית מערכות ממוחשבות מבוססות אינטרנט הרצאה 5 – node.js



Working with the command line interface (CLI)

- To make git bash the default terminal in VSCode-
 - view→command palette→select default profile→git bash
 - In the terminal- a \$ sign will appear prior to the command input
- The CLI provides better control over the operation system and the computer's hardware
- Common commands:
 - \$ls- prints the content of the current directory
 - \$cd- change directory
 - \$mkdir- opens a new directory in the current location
 - \$touch Text.txt- opens a new file "Text.txt"
 - Etc.



Backend Development

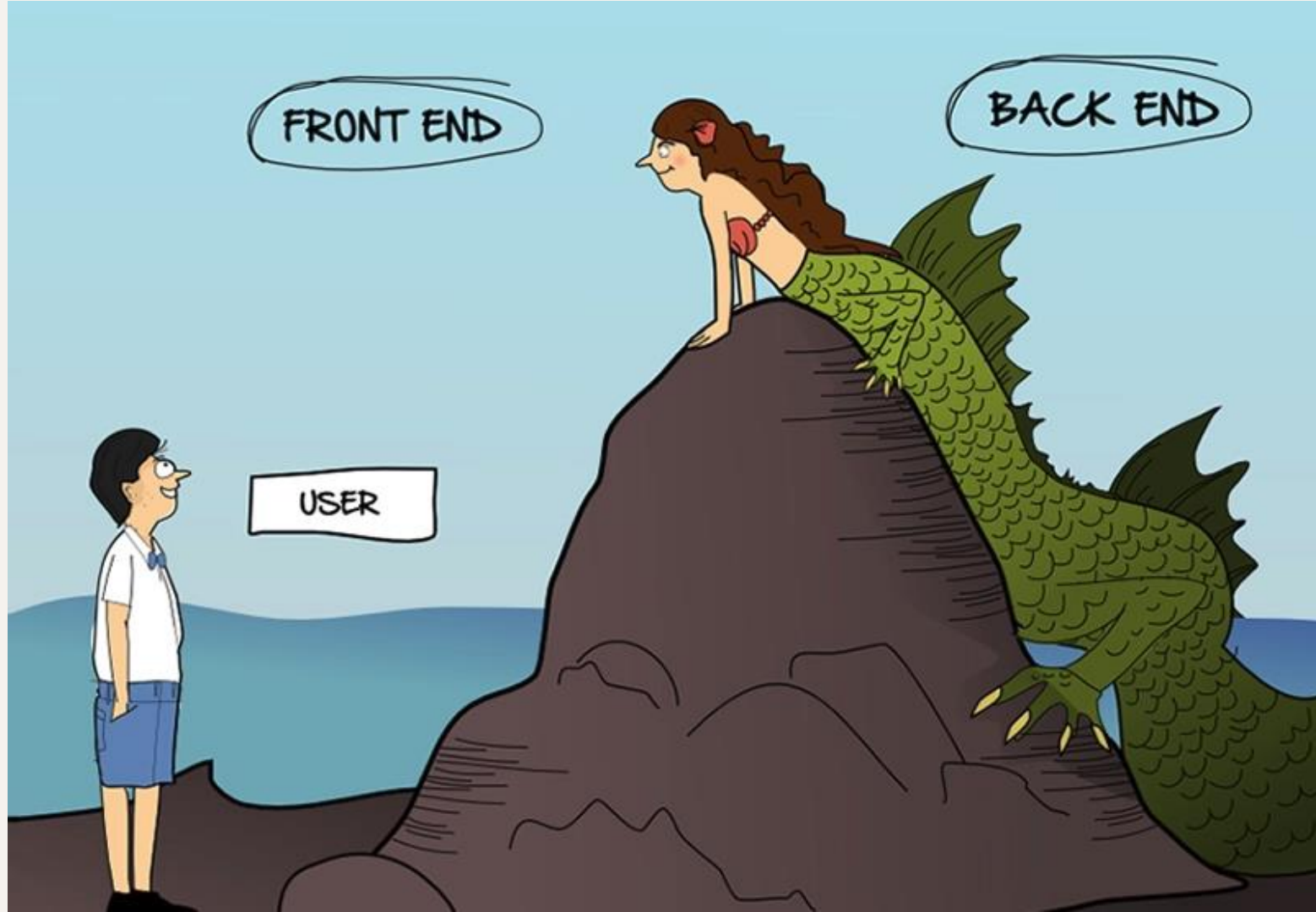
Backend development

- Three main components:
 - **Server (the physical part)**
 - **Application (logic)**
 - **Database**
- The server- internet server will be open 24/7, listening to clients' requests.
- The application- a logic that runs on the server and determines how the server will respond the clients' requests (e.g., 404 not found)
- The database- not mandatory, but- as long as the website need to save information, a database will be needed

FRONT END

BACK END

USER



The Node.js logo, featuring the word "node" in a stylized, lowercase, black font. The letter "o" is replaced by a green, three-dimensional cube. A small green dot is positioned to the right of the letter "e".

node



Node.js

What is Node.js?

- Node.js is an **asynchronous event-driven JavaScript runtime environment.**
 - runtime environment- allows to run JavaScript code outside the browser based on V8 google engine
 - asynchronous event-driven- the code will run based on event and in an asynchronous manner
- Node.js runs on various platforms (Windows, Linux, Unix, Mac OS X, etc.)
- Node.js is an a free open-source server environment

Synchronous Vs. asynchronous

- Synchronous- pick up order → go to kitchen to pass the order → wait for the food to be ready → serve the food to the customer → continue to the next customer
- Asynchronous- pick up order → go to kitchen to pass the order → continue to the next customer → when food is ready- serve it to the relevant customer

A common task for a server: to open a file on the server and return the content to the client

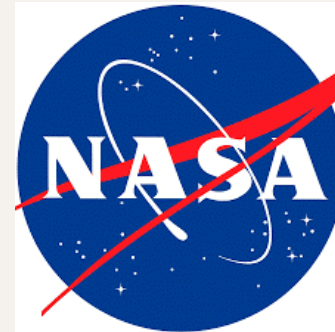
PHP / ASP

1. Sends the task to the computer's file system.
2. Waits while the file system opens and reads the file.
3. Returns the content to the client.
4. Ready to handle the next request.

Node.js

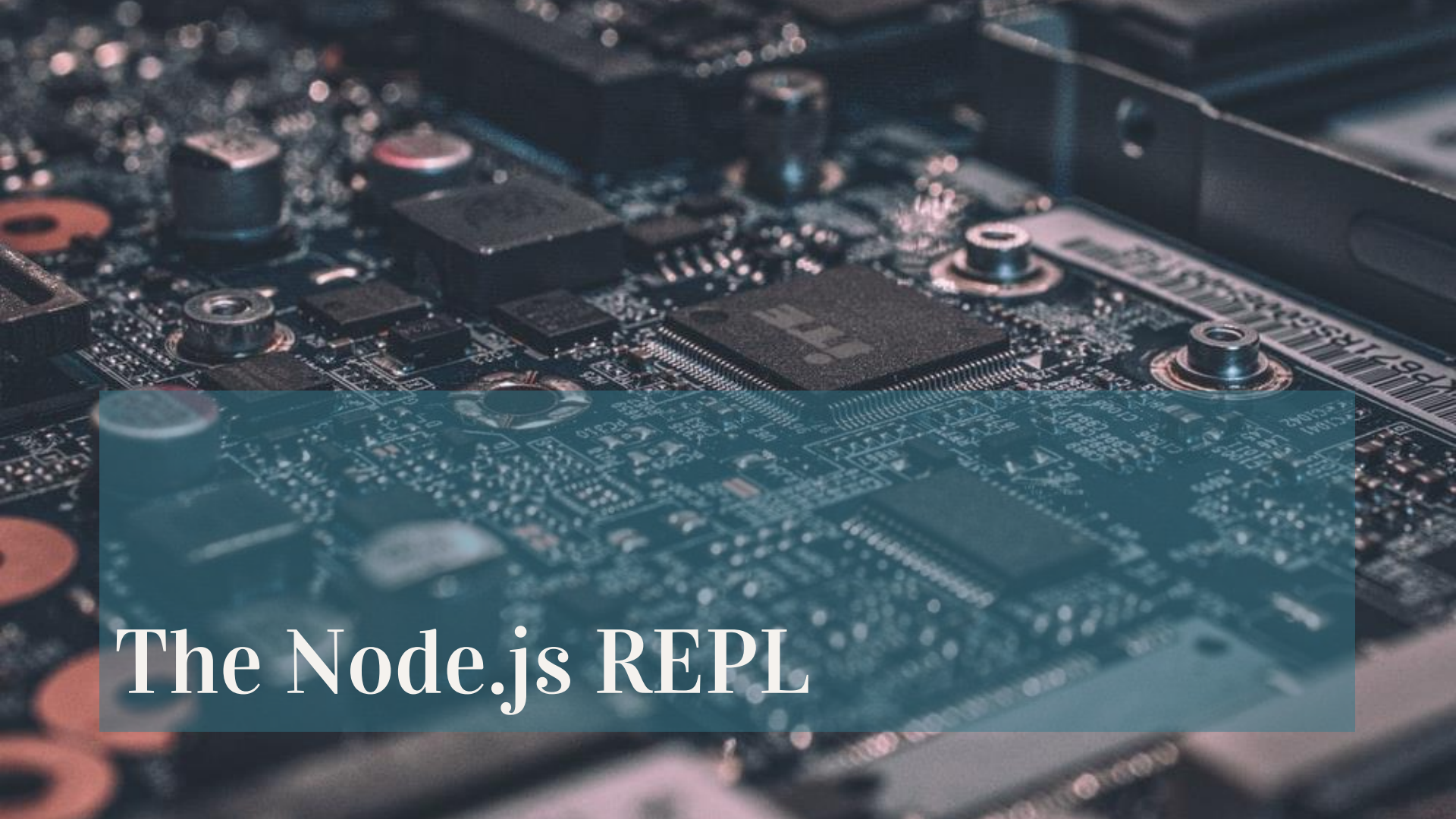
1. Sends the task to the computer's file system.
2. Ready to handle the next request.
3. Returns the content to the client.

Companies using Node.js



What Can Node.js Do?

- Node.js can **generate dynamic page** content
- Node.js can **create, open, read, write, delete, and close files** on the server
- Node.js can **collect 'form'** data
- Node.js can **add, delete, modify data** in your database



The Node.js REPL

What is the REPL?

- REPL- Read, Eval, Print, Loop
- The REPL is the environment in which we can insert inputs, the computer reads it, evaluated it, prints the result, and loop again
- Enter the Node.js REPL:
 - In the terminal type node.

Working with the REPL

- In the terminal type node.

```
$ node
Welcome to Node.js v20.10.0.
Type ".help" for more information.
> a=2
2
> b=a+5
7
> for (var i=0;i<2;i++){console.log(i);}
0
1
undefined
> 
```


Executing JavaScript file using the REP

- In the terminal type node+fileName.

```
JS 1 REPL.js X
JS 1 REPL.js
1 console.log("Hello from Node.js");
2 //now run the code using the REPL in the terminal
3 // type node + filename (or node+ using alt to autofill the filename)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

itzik@LAPTOP-F5BD2N7D MINGW64 ~/OneDrive/חלום/הדובע/הדובע/WEB/ה
$ node 1\ REPL.js
Hello from Node.js

itzik@LAPTOP-F5BD2N7D MINGW64 ~/OneDrive/חלום/הדובע/הדובע/WEB/ה
$
```



Getting started- deep dive into
the ocean

Agenda

- Uploading a web-page to a remote server, and serving it by request
- Steps:
 1. Create an html page
 2. Create a node.js server
 3. Run web-site locally
 4. Deploy a single page web-site using git and heroku

Preliminary step 1

- open a new directory in your computer \create a new repository in github and clone it to your local machine
- Example:
- myFirstWebSite repository cloned into local as myFirstWebSite folder

Step 1: create an html page

- Create a new html page and save it in a new folder.
- Example:
myFirstWebSite.html file,
saved in myFirstWebSite
folder

```
<!DOCTYPE html>
<html lang="en">
<head>
  <title>My First Web Site</title>
</head>
<body>
  <h1>Hello World</h1>
  <p>This is my first non-local web-site</p>
</body>
</html>
```

Step 2: create a server

- In the root folder create a node.js server (a JavaScript file).
- Example:
myFirstServer.js file,
saved in myFirstWebSite
folder

```
const http = require('http');  
  
http.createServer(function(req, res) {  
  res.write('Hello World!');  
  res.end();  
}).listen(8080);
```

Step 2: create a server

This example simple server:

1. utilizes the http module, which enables it with http functionality
2. Uses createServer method to define a simple function that has an http request and a http response: the string “hello world”
3. Set to listen to local port 8080

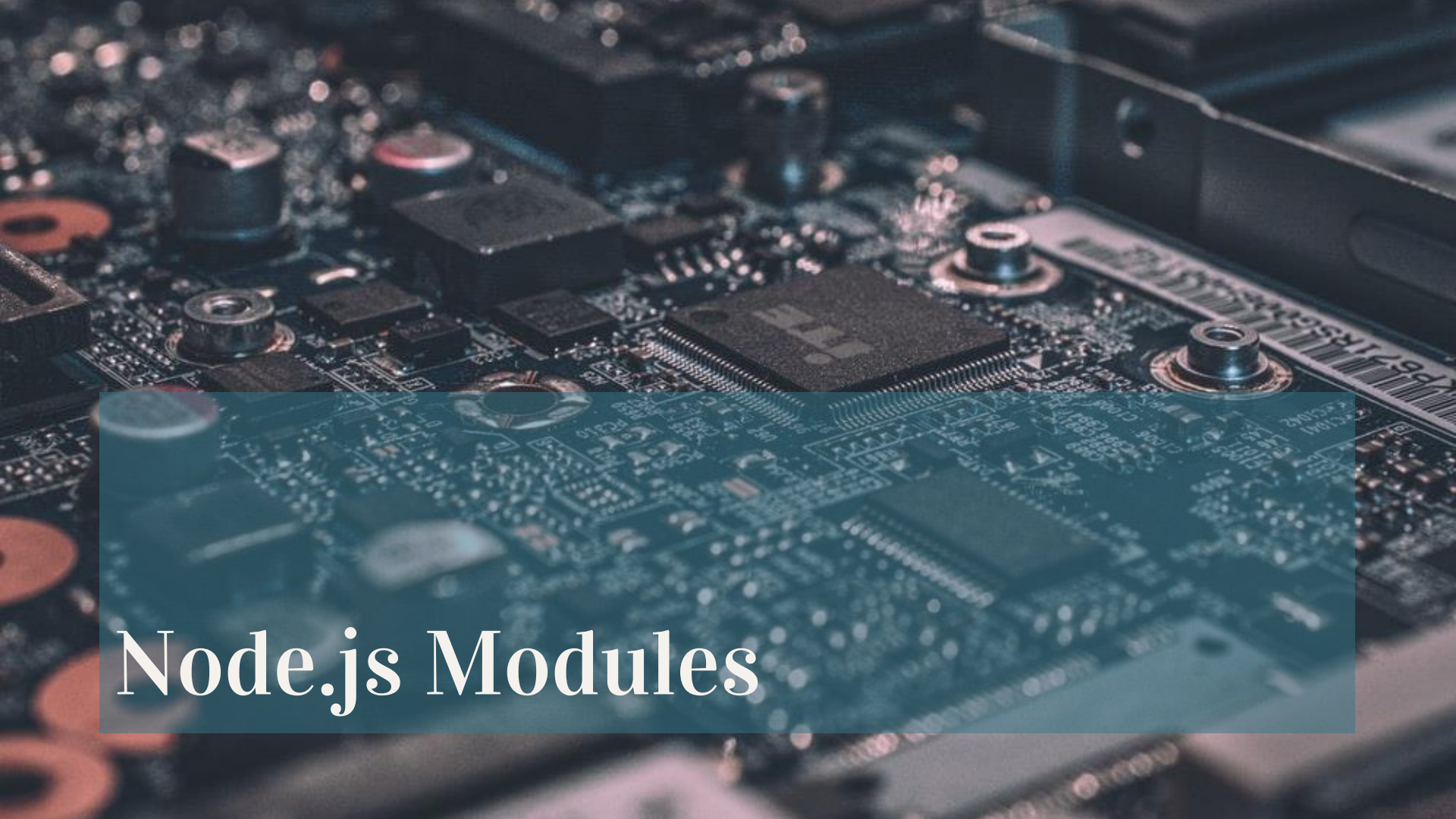
```
const http = require('http');  
  
http.createServer(function(req, res) {  
  res.write('Hello World!');  
  res.end();  
}).listen(8080);
```


Check server locally

- Let's check out our website locally using the command line interface/cmd
 - Use the REPL by typing node+filename In the terminal
 - Use the cmd opened from “myFirstWebsite” directory in your computer

Check local host

- Now, your computer works as a server!
- If anyone tries to access your computer on port 8000, they will get a "Hello World!" message in return!
- Open your internet browser, and type in the url: <http://localhost:8000>
- To kill the server, type ctrl+c in the terminal



Node.js Modules

What is a Module in Node.js?

- Consider modules to be the same as JavaScript libraries.
- A set of functions you want to include in your application.
 - Built-in Modules
 - Create Your Own Modules

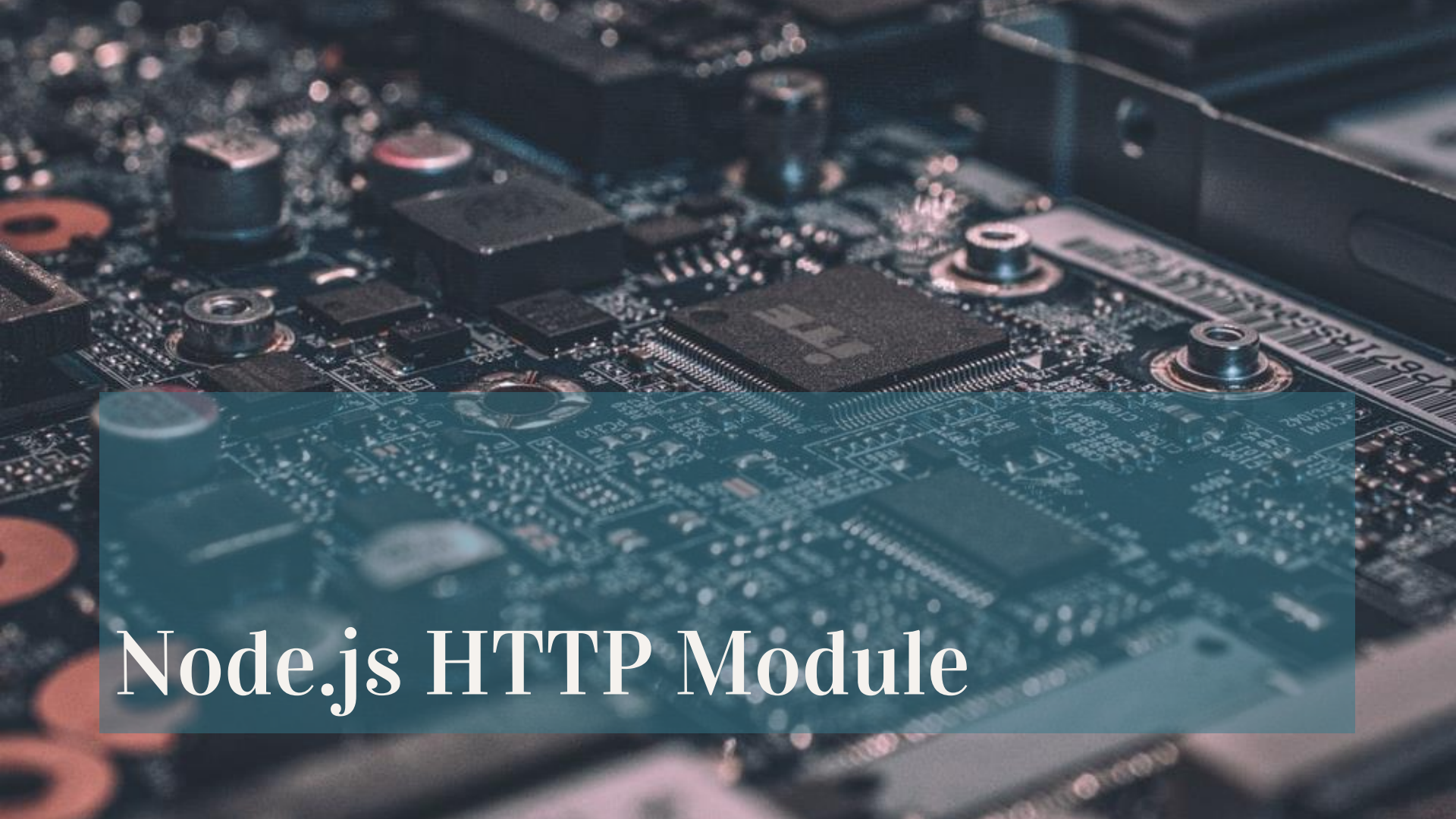
Node.js Main Module

- Node.js **File System Module**
- Node.js **HTTP Module**
- Node.js **URL Module**
- Etc.

Include Modules

- To include a module, use the `require()` function with the name of the module:

```
const http = require('ModuleName');
```

Node.js HTTP Module

HTTP Module

- Node.js has a built-in module called HTTP.
- It allows Node.js to transfer data over the Hyper Text Transfer Protocol (HTTP).
- The HTTP module can create an HTTP server that **listens to server ports** and gives a response back to the client.
- Use the `createServer()` method to create an HTTP server

Create an HTTP Server

```
const http = require('http');  
  
//create a server object:  
http.createServer(function (req, res) {  
  res.write('Hello World!'); //write a response to the client  
  res.end(); //end the response  
}).listen(8080); //the server object listens on port 8080
```

- The function passed into the `http.createServer()` method, will be executed when someone tries to access the computer on port 8080.

Add an HTTP Header

- An HTTP header is a field of an HTTP request or response that passes additional context and metadata about the request or response.
- If the response from the HTTP server is supposed to be displayed as HTML, you should include an HTTP header with the correct content type.

```
const http = require('http');
http.createServer(function (req, res) {
  res.writeHead(200, {'Content-Type': 'text/html'});
  res.write('Hello World!');
  res.end();
}).listen(8080);
```

Add an HTTP Header

- The first argument of the `res.writeHead()` method is the status code (200 means that all is OK)
- The second argument is an object containing the response headers.

```
var http = require('http');  
http.createServer(function (req, res) {  
  res.writeHead(200, {'Content-Type': 'text/html'});  
  res.write('Hello World!');  
  res.end();  
}).listen(8080);
```

Status codes:

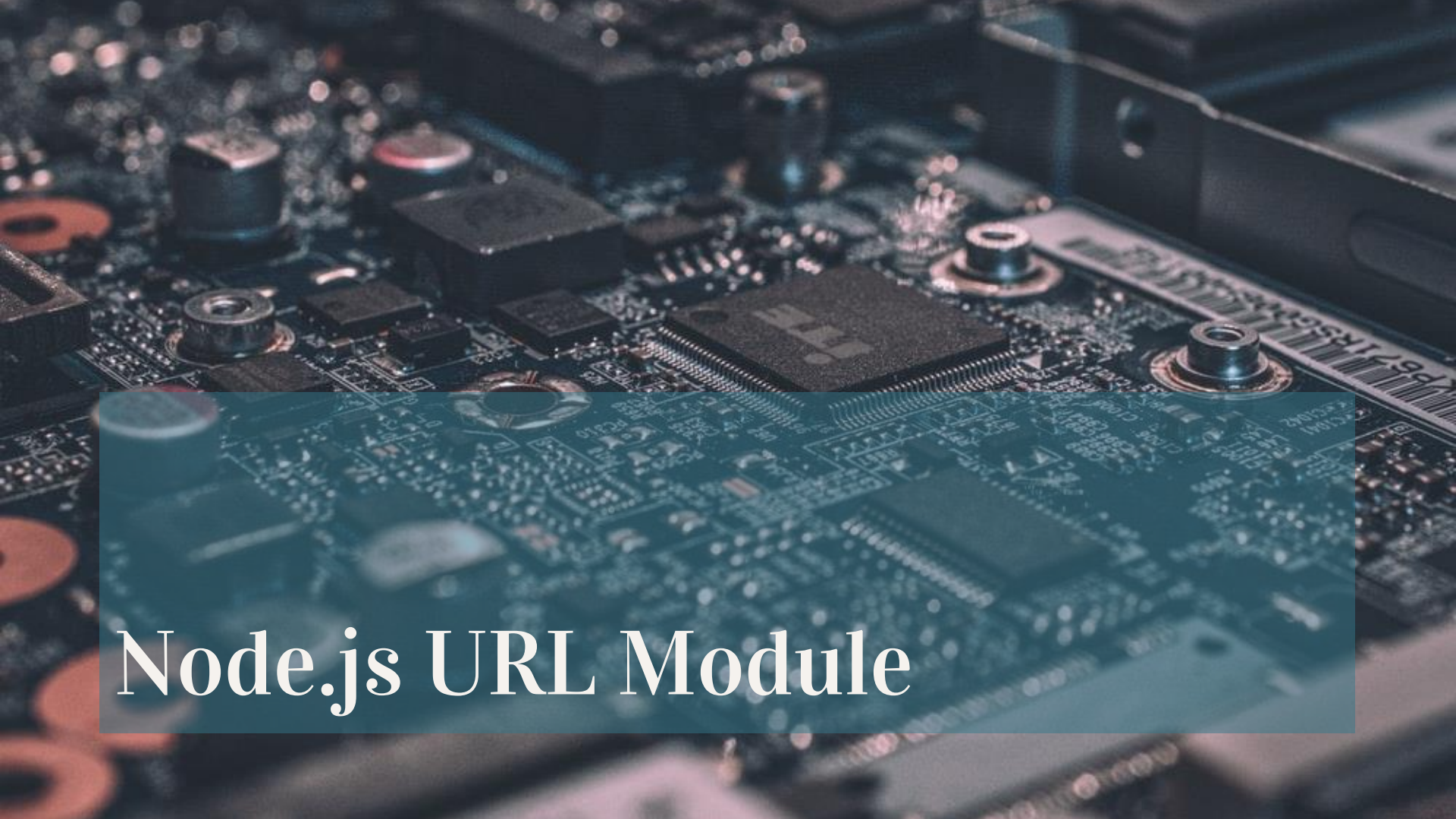
1. Informational responses (100 – 199)
2. Successful responses (200 – 299)
3. Redirection messages (300 – 399)
4. Client error responses (400 – 499)
5. Server error responses (500 – 599)

Read the Query String

- The function passed into the `http.createServer()` has a **req** **argument** that represents the request from the client, as **an** **object** (`http.IncomingMessage` object).
- This object has a property called "url" which holds the part of the url that comes after the domain name:

```
var http = require('http');  
http.createServer(function (req, res) {  
  res.writeHead(200, {'Content-Type': 'text/html'});  
  res.write(req.url);  
  res.end();  
}).listen(8080);
```





Node.js URL Module

URL Module

- The URL module splits up a web address into readable parts.
- Reminder: every URL is built from protocol, domain, subdomain, and parameters.
- To include the URL module, use the require() method:

```
const url = require('url');
```

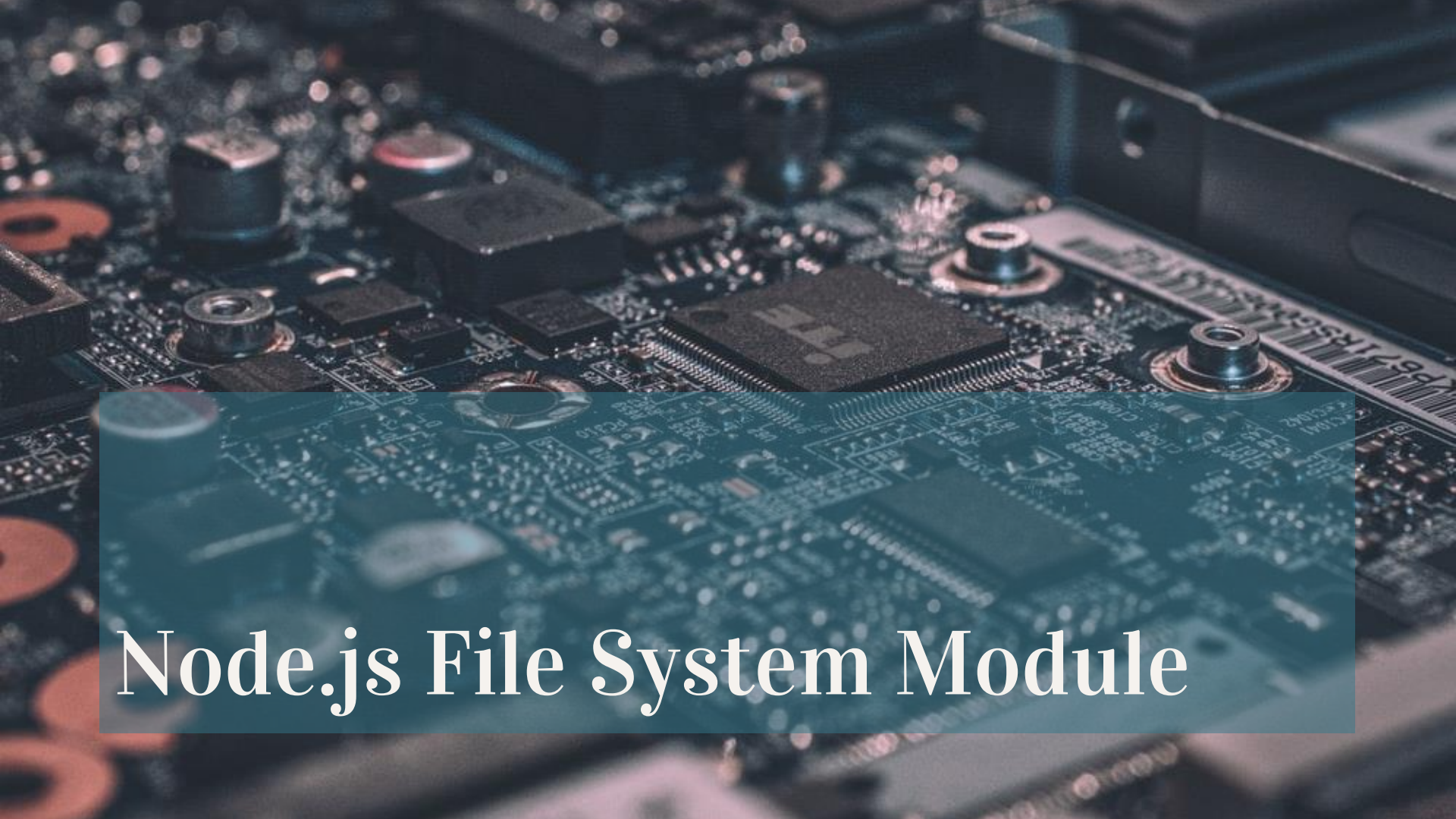
URL Module

- Parse an address with the `url.parse()` method, and it will return a URL object with each part of the address as properties

```
const url = require('url');
var adr = 'http://localhost:8080/default.html?year=2017&month=february';
var q = url.parse(adr, true);

console.log(q.host); //returns 'localhost:8080'
console.log(q.pathname); //returns '/default.html'
console.log(q.search); //returns '?year=2017&month=february'

var qdata = q.query; //the query comes after the ?- returns an object: { year: 2017, month: 'february' }
console.log(qdata.month); //returns 'february'
```

Node.js File System Module

File System Module

- The Node.js file system module allows you to work with the file system on your computer.
- Why do we need it?
- To include the File System module, use the require() method:

```
const fs = require('fs');
```

Node.js File System

Common use for the File System module:

1. Read files
2. Create files
3. Update files
4. Delete files
5. Rename files

Write file

```
var fs = require('fs');  
fs.writeFile('Text.txt', 'this is a new content', (err)=> {  
    if (err) throw err;  
    Console.log("file has been saved!");  
})
```

Read Files

- The fs.readFile() method is used to read files on your computer.
- Assume we have the following HTML file:

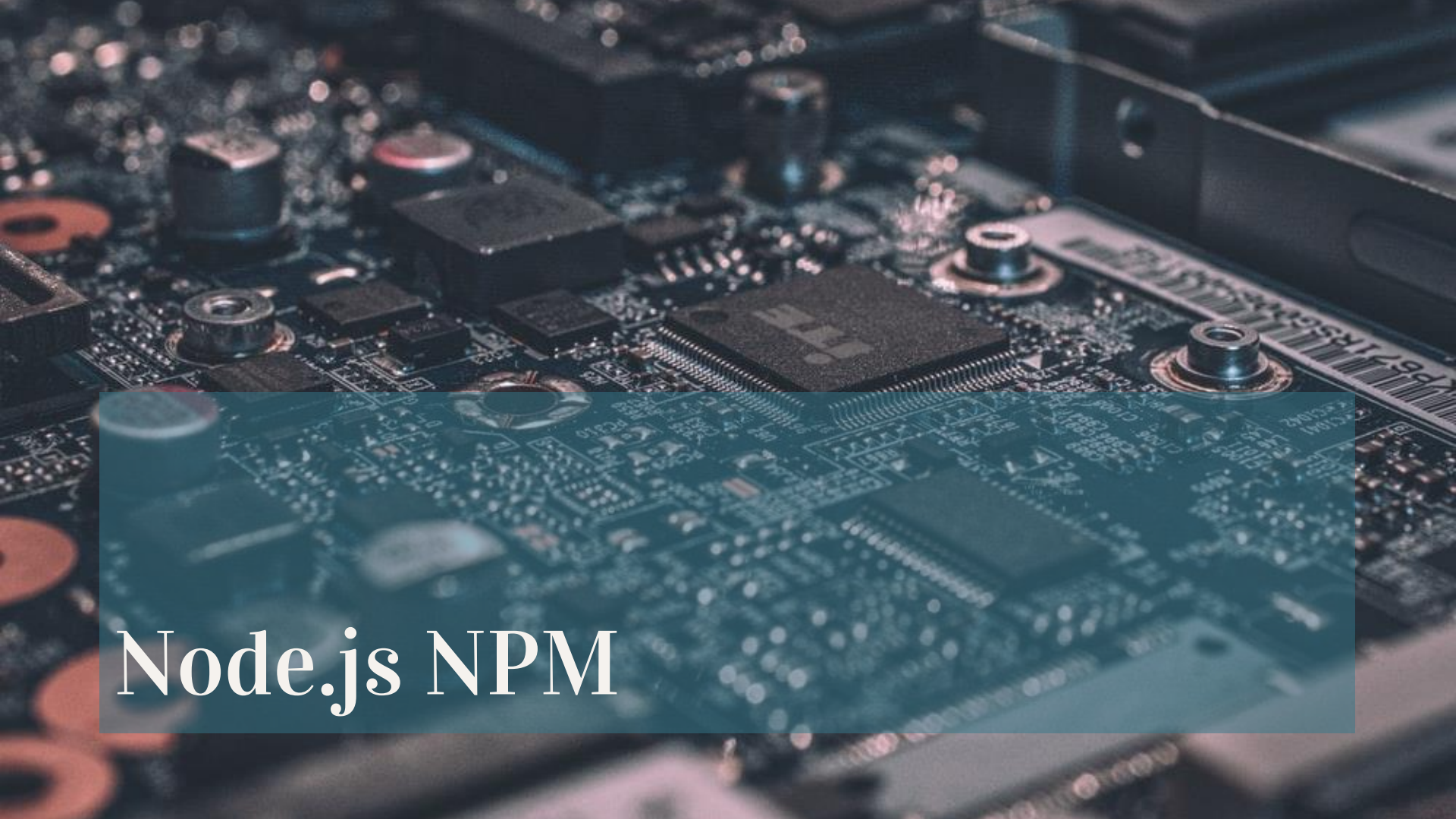
```
<html>  
<body>  
<h1>My Header</h1>  
<p>My paragraph.</p>  
</body>  
</html>
```

Read Files

- Create a Node.js file that reads the HTML file, and return the content:

```
var http = require('http'); // using http module
var fs = require('fs'); //using fs module

http.createServer(function (req, res) { //creating a server
  //Open a file on the server and return its content:
  fs.readFile('index.html', function(err, data) {
    if (err){
      res.writeHead(404, {'Content-Type': 'text/plain'});
      res.write ('404 Not Found');
      return res.end();
    }
    res.writeHead(200, {'Content-Type': 'text/html'}); //the content is html therefore
    the content type is text/html
    res.write(data); //the server will respond with the data from the html file
    return res.end();
  }); //end of readFile method
}).listen(8081);
console.log("server is running on port 8081"); //end of createServer method
```



Node.js NPM

What is NPM- Node Package Manager?

- NPM is a package manager for Node.js package.
- www.npmjs.com hosts thousands of free packages to download and use.
- The NPM program is installed on your computer when you install Node.js
- NPM is already ready to run on your computer!

What is a Package?

- A package in Node.js contains all the files you need for a module.
- Modules are JavaScript libraries you can include in your project.

Download a Package

- Downloading a package is very easy.
- Open the command line interface (or terminal in VSCode) download the package you want.
- Download a package called "sillyname":

```
C:\Users\Your Name>npm install sillyname
```

```
-----!!!OR!!!-----
```

```
C:\Users\Your Name>npm i sillyname
```

Example using 'sillyname' module

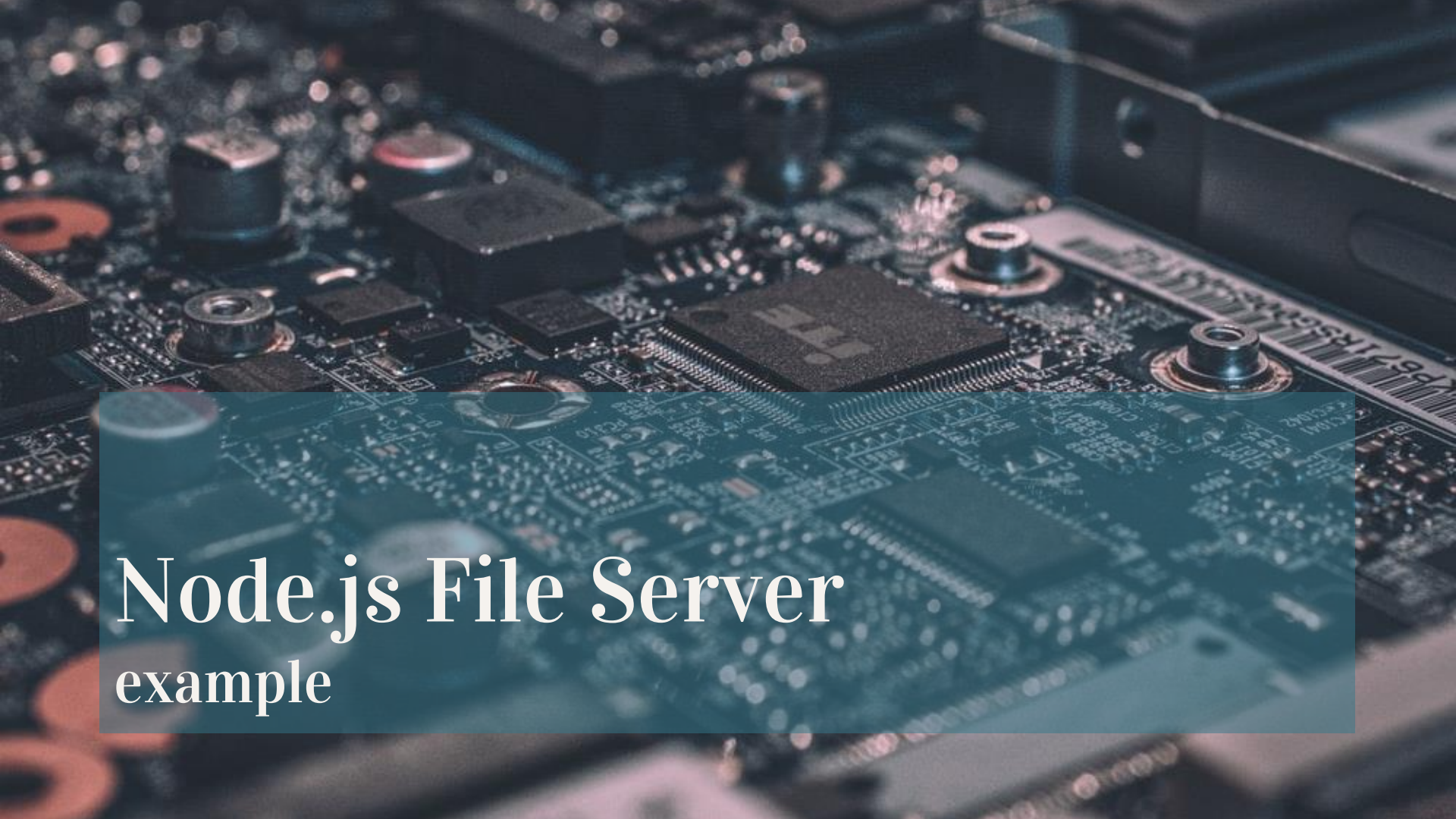
```
//https://www.npmjs.com/package/sillyname
```

```
var generateName = require('sillyname');  
var sillyName = generateName();  
console.log(`my name is ${sillyName}`);
```

```
itzik@LAPTOP-F5BD2N7D MINGW64 ~  
$ node 4\ NPMSillyname.js  
my name is Plumcollar Hide
```

Exercise- qr code generator

- Using the npm modules:
 - Inquirer (<https://www.npmjs.com/package/inquirer#methods>)
 - Qr-image (<https://www.npmjs.com/package/qr-image>)
- You need to:
 1. Use the inquirer npm package to get user input.
 2. Use the qr-image npm package to turn the user entered URL into a QR code image.



Node.js File Server

example

Walk me through example

- Now we know how to:
 1. Build html pages
 2. make Node.js behave as a file server
- Let us combine the two, and serve the file requested by the client.
- In the following example you will create 2 html files and 1 node.js file and save them all in the same folder.

Create 2 html files

- summer.html

```
<!DOCTYPE html>
<html>
<body>
<h1>Summer</h1>
<p>I love the sun!</p>
</body>
</html>
```

- winter.html

```
<!DOCTYPE html>
<html>
<body>
<h1>Winter</h1>
<p>I love the snow!</p>
</body>
</html>
```

Create server

- Create a Node.js file that
 1. opens the requested file
 2. returns the content to the client.
 3. If anything goes wrong, throw a 404 error

```
const http = require('http');
const url = require('url');
const fs = require('fs');

http.createServer(function (req, res) {
  var q = url.parse(req.url, true);
  var filename = "." + q.pathname;
  fs.readFile(filename, function(err, data) {
    if (err) {
      res.writeHead(404, {'Content-Type': 'text/html'});
      return res.end("404 Not Found");
    }
    res.writeHead(200, {'Content-Type': 'text/html'});
    res.write(data);
    return res.end();
  });
}).listen(8080);
```


Run the server

- Initiate the file in your command line

```
C:\Users\Your Name>node demo_fileserver.js
```

Check your browser

- Check in your browser the following URLs:
 1. <http://localhost:8080/summer.html>
 2. <http://localhost:8080/winter.html>